ONLINE CLOTHING STORE APP

PREPARED BY: **AHMED IRSHAD HUSSAIN**

(sardarahmed.irshad2@gmail.com)

ONLINE CLOTH STORE APPLICATION

The Online Clothing Store Application that I developed, allows customers to view products, add items to the cart, and place orders. The backend is built using **Node.js, Express.js, MySQL**, and provides REST APIs for product management, cart operations, and order processing. The frontend uses **HTML, CSS, JavaScript**, and interacts with the backend API.

This document focuses on:

- Explaining **exception handling** implemented in the system.
- Designing a **detailed Test Plan**.
- Performing **Manual, Unit, Functional and Integration testing.**
- Adding screenshots as evidence that test cases executed successfully.

EXCEPTION HANDLING:

Our backend contains structured error handling using:

- **try–catch blocks**

  Used to safely run database operations without application crash.

```
// ✅ Checkout Route
app.post("/checkout", async (req, res) => {
  const { cartItems } = req.body;

  if (!cartItems || cartItems.length === 0) {
    return res.status(400).json({ success: false, message: "Cart is empty" });
  }

  try {
    await safeUpdateStock(async () => {
      db.beginTransaction((err) => {
        if (err) throw err;

        console.log("🛒 Checkout request received:", cartItems);

        const ids = cartItems.map((i) => i.id);
        db.query(
          "SELECT id, stock FROM products WHERE id IN (?)",
          total,
        });
      });
    })
    .catch((err) => {
      console.error("Stock update error:", err);
      db.rollback();
      res.status(500).json({ success: false, message: "Stock update failed" });
    });
}
```

- **Express Global Error Handler**

```
  });
});

app.use((err, req, res, next) => {
  console.error("🔥 Server Error:", err.stack);
  res.status(500).json({ error: "Internal Server Error" });
});

app.listen(PORT, () => console.log(`🚀 Server running on port ${PORT}`));
```

- **Database Error Handling:**

Catches:

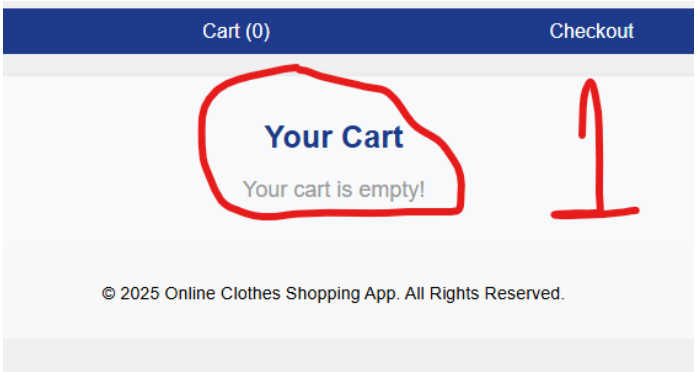
- MySQL connection failure
- Invalid SQL queries
- Duplicate data insertions
- Missing/invalid request fields

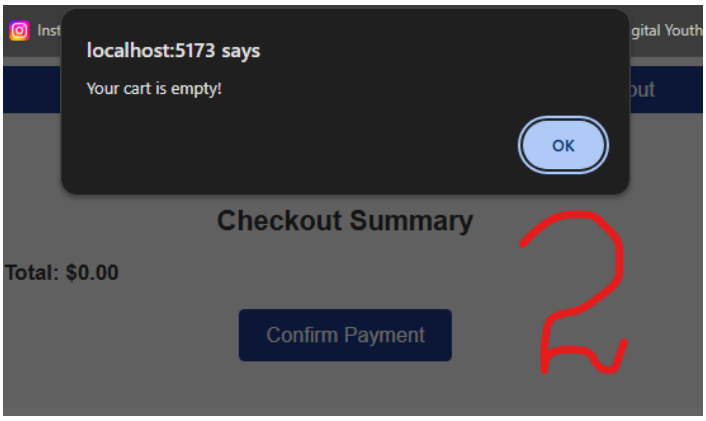
```
end > routes > JS products.js > ...
import express from "express";
import db from "../db.js";

const router = express.Router();


try {
  const [rows] = await db.query("SELECT * FROM products");
  res.json(rows);
} catch (error) {
  res.status(500).json({ error: "Failed to fetch products" });
}
```

- **<u>Validation Errors (Customs):</u>**

During checkout:

- If product stock is insufficient → return error `"Out of Stock"`
- If cart array is empty → `"Cart cannot be empty"`
- If API receives invalid parameter → `"Invalid Request"`





As you can see in picture 1, cart is empty and when we proceed to checkout, picture 2, message popped up regarding "Empty Cart"

- **<u>Concurrency Protection:</u>**

During stock deduction:

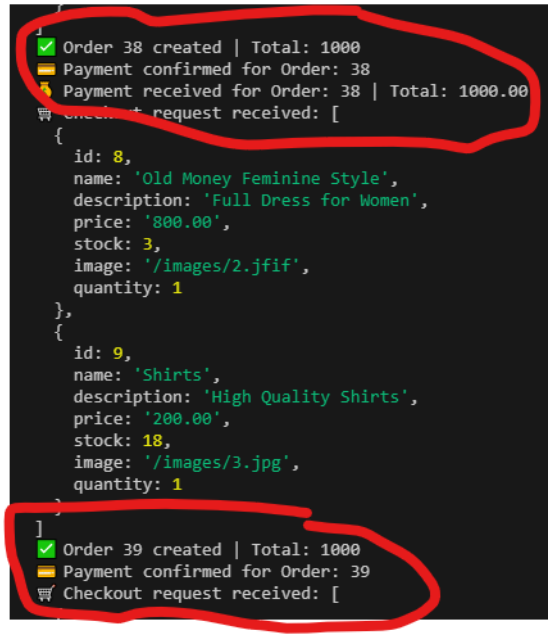
- Stock is only reduced after confirming availability.
- Prevents race conditions & overselling.

<u>Example 1:</u>

```
  }
]
✅ Order 43 created | Total: 400
💳 Payment confirmed for Order: 43
💰 Payment received for Order: 43 | Total: 400.00
🛒 Checkout request received: [
  {
    id: 9,
    name: 'Shirts',
    description: 'High Quality Shirts',
    price: '200.00',
    stock: 14,
    image: '/images/3.jpg',
    quantity: 1
💰 Payment received for Order: 44 | Total: 400.00
💰 Payment received for Order: 44 | Total: 400.00
💰 Payment received for Order: 44 | Total: 400.00
🛒 Checkout request received: [
  {
    id: 9,
    name: 'Shirts',
    description: 'High Quality Shirts',
    price: '200.00',
    stock: 13,
    image: '/images/3.jpg',
    quantity: 1
  },
  {
    id: 10,
    name: 'Jeans',
    description: 'High Quality Jeans',
    price: '200.00',
    stock: 19,
    image: '/images/4.jfif',
  }
]
✅ Order 45 created | Total: 400
💳 Payment confirmed for Order: 45
💰 Payment received for Order: 45 | Total: 400.00
```

When order 44 (1) was created, the stock of shirts were 14, when order 45 created (2), the stock of shirts were decremented to 13.

Example 2:



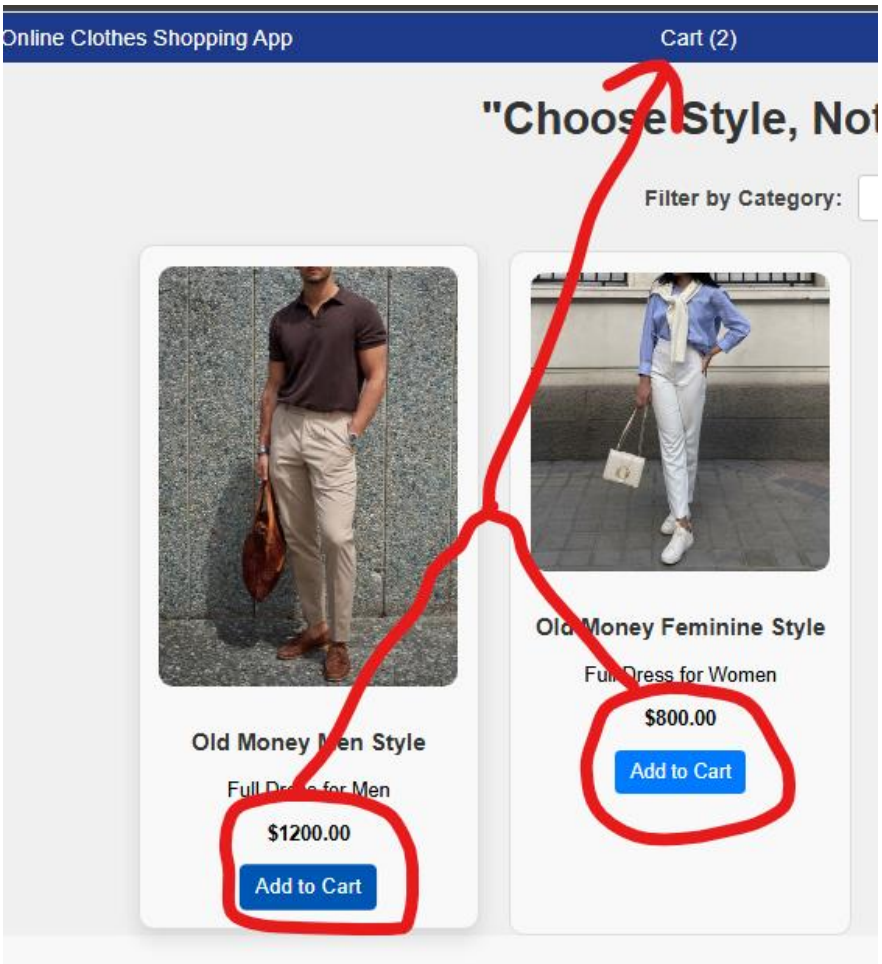
Terminal logs showing **multiple checkout requests** (Order 38, Order 39, etc.) running concurrently.

MANUAL TESTING:

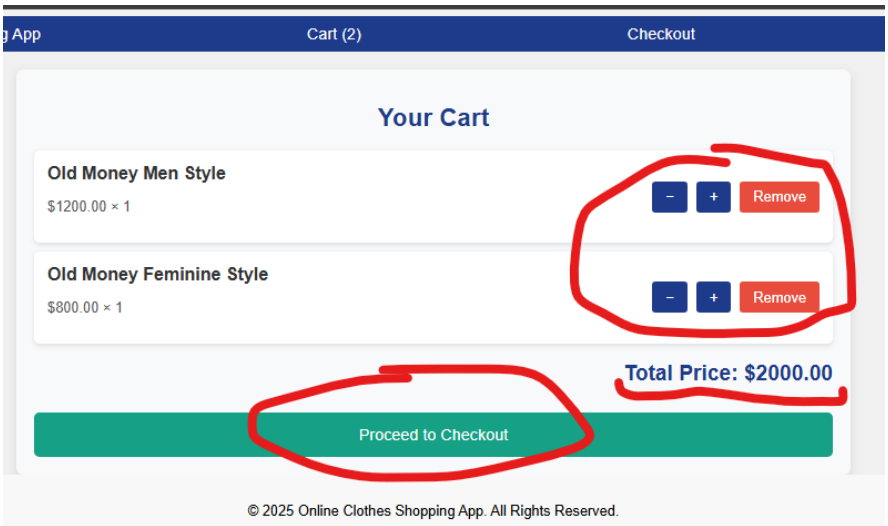
Manual testing checks the system behavior through user interactions.

Test Areas Covered

- Product listing
- Product detail page
- Add to cart
- Remove from cart
- Place order
- Stock reduction
- Error messages
- Database updates



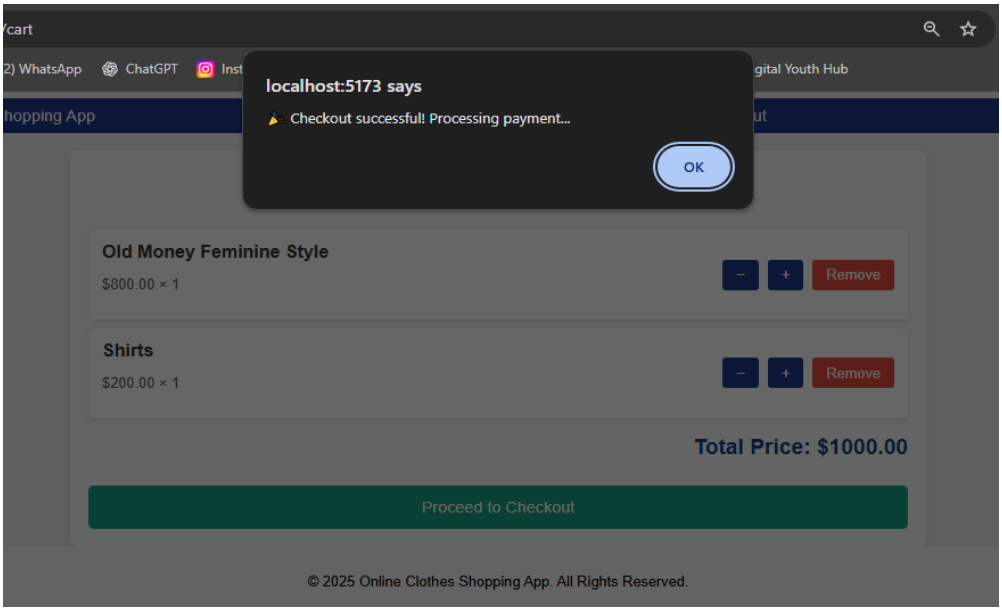
As customer adds 2 products into the cart so it will be incremented by 2.

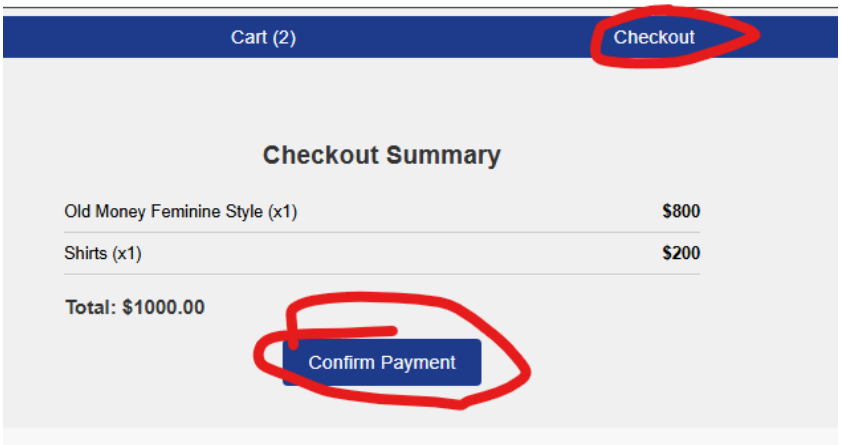


This is the cart page, where you add(+), substract(-) or remove the whole product

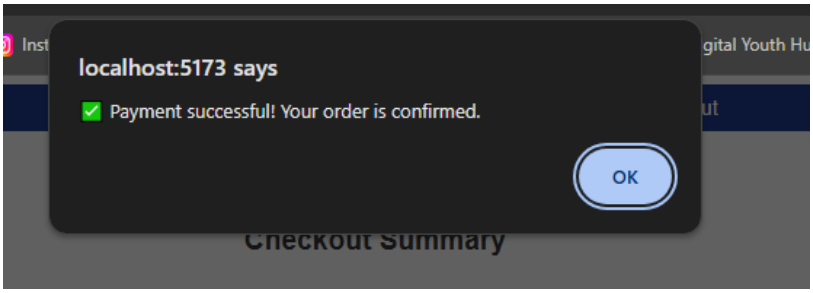
.

I also click on the “Proceed to Checkout” button





This is checkout page where the selected products are now showing for the “payment confirmation”



after clicking “confirm payment”, message popped up of order placement!

FINAL REMARKS:

The client is fully functional and interacts with the server asynchronously using REST API calls. Each client request (fetch products, checkout, payment confirmation) runs independently and does not block other users.

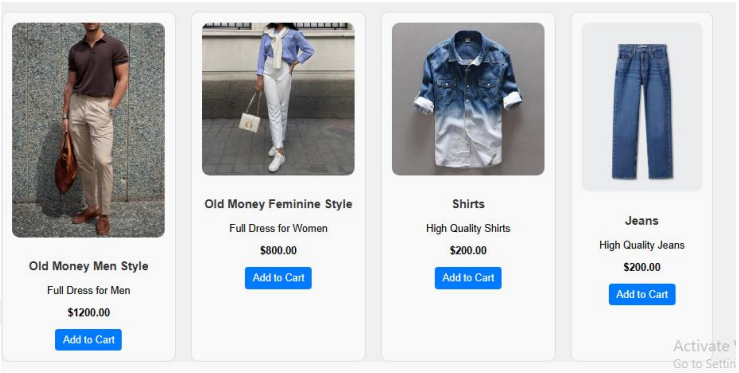
UNIT TESTING:

Unit Testing 1: Fetch Products:

Objective: To ensure products load correctly.

No	Test Case	Input Data	Expected Output	Result
1	Fetch all products	GET `/products`	Returns list of products	Pass
2	Force DB failure	Corrupt query	Returns error “Failed to fetch products”	Pass

```
  product_id INT,
  quantity INT,
  FOREIGN KEY (order_id) REFERENCES orders(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
INSERT INTO products (name, description, price, stock, image)
VALUES
('Old Money Men Style', 'Full Dress for Men', 1200.00, 10, '/images/1.jpg'),
('Old Money Feminine Style', 'Full Dress for Women', 800.00, 15, '/images/2.jfif'),
('Shirts', 'High Quality Shirts', 200.00, 25, '/images/3.jpg'),
('Jeans', 'High Quality Jeans', 200.00, 25, '/images/4.jfif');

SELECT * FROM products;
DELETE FROM order_items WHERE product_id < 7;

DELETE FROM products WHERE id < 7;
```



Backend (MySQL) has 4 products and frontend is successfully displaying the products!

Unit Testing 2: Add to Cart:

Objective: Ensure items are added safely.

No	Test Case	Input	Expected Output	Result
1	Add existing product	Product ID = 8	Added to cart successfully	Pass
2	Add non-existing product	Product ID = 999	“Product not found”	Pass

Unit Testing 3: Checkout Stock Validation:

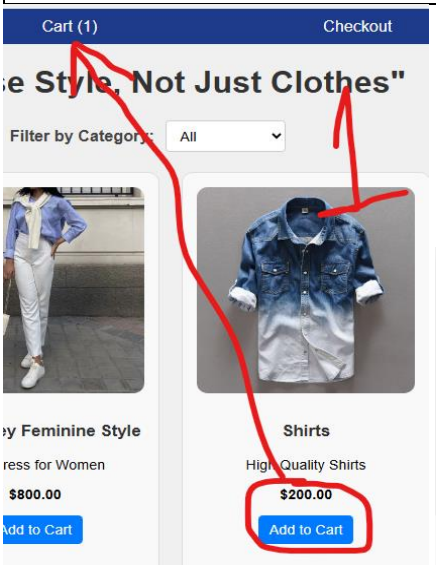
Objective: Ensure stock is checked before deduction.

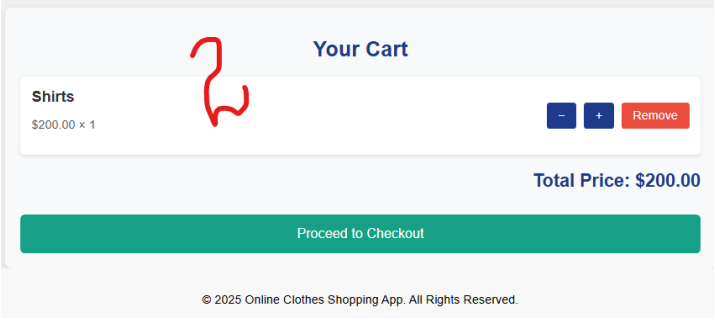
No	Test Case	Cart Input	Expected Output	Result
1	Valid stock	Qty < Stock	Order successful	Pass
2	Insufficient stock	Qty > Stock	Error: “Out of Stock”	Pass

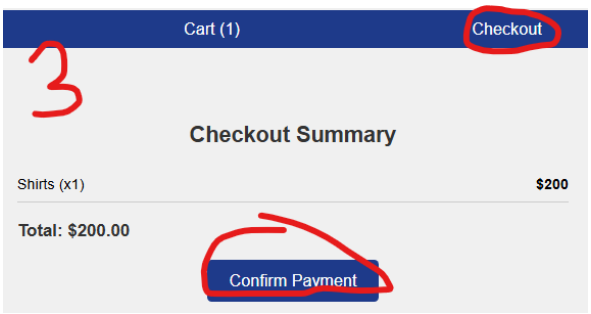
FUNCTIONAL TESTING:

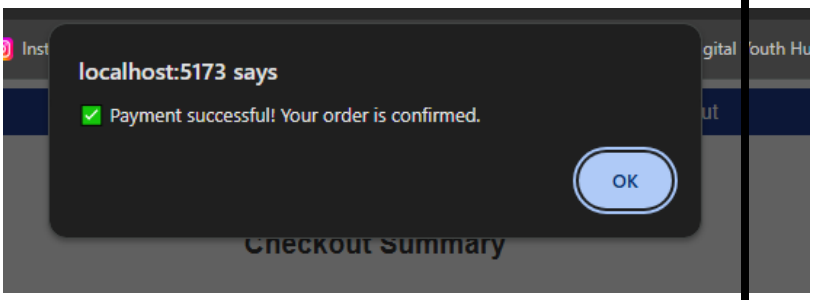
Functional Testing 1: Complete Order Flow:

No	Action	Input	Expected Behavior	Result
1	Select product	Click product card	Product details shown	Pass
2	Add to cart	Qty = 1	Cart updated	Pass
3	View cart	Click cart	Items display correctly	Pass
4	Checkout	Place Order	Order stored in DB & stock reduced	Pass



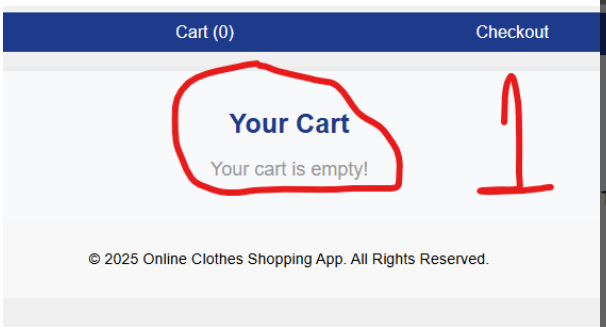


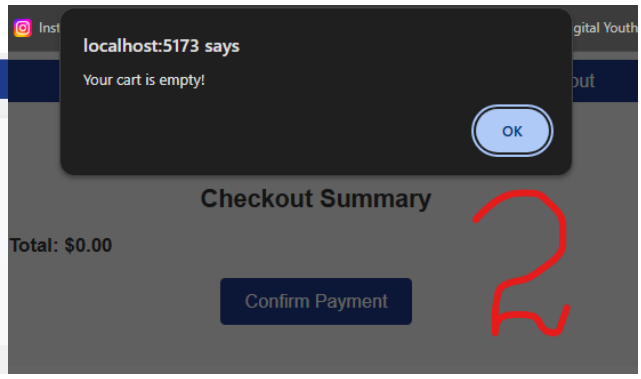




Functional Testing 2: Error Handling:

No	Action	Test Input	Expected Behavior	Result
1	Checkout with empty cart	[]	"Cart is empty"	Pass
2	Checkout with huge qty	qty = 1000	"Out of Stock"	Pass



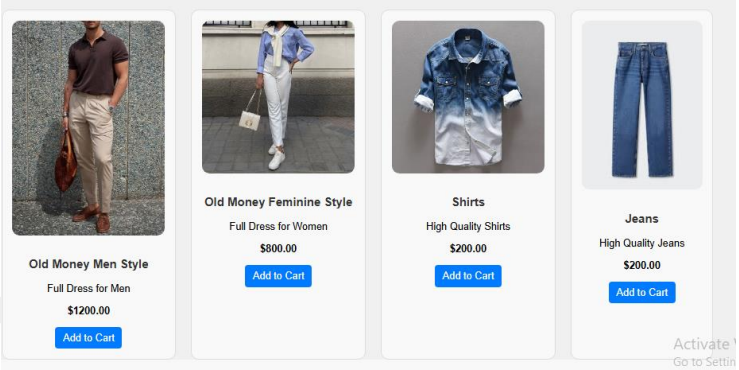


INTEGRATION TESTING:

Integration Testing 1: Product Display + API + DB:

No	Test Case	Process	Expected Result	Result
1	Load products	Frontend → GET /products → MySQL	Products display on UI	Pass

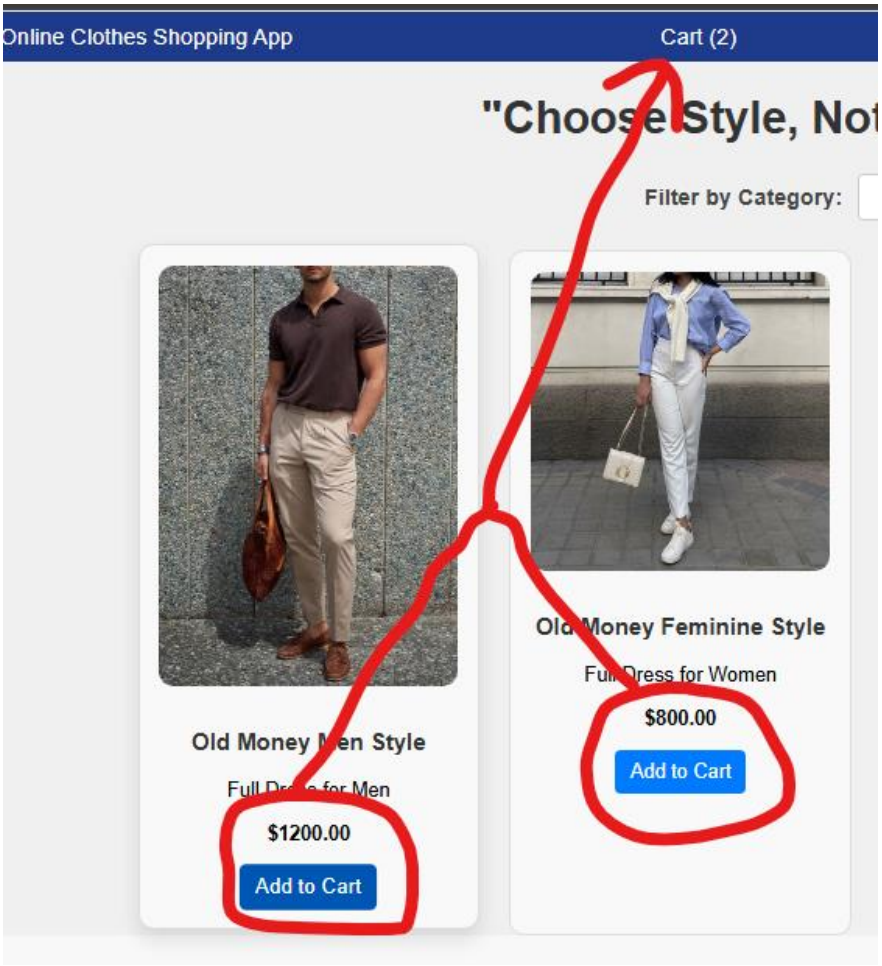
```
  product_id INT,
  quantity INT,
  FOREIGN KEY (order_id) REFERENCES orders(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
INSERT INTO products (name, description, price, stock, image)
VALUES
('Old Money Men Style', 'Full Dress for Men', 1200.00, 10, '/images/1.jpg'),
('Old Money Feminine Style', 'Full Dress for Women', 800.00, 15, '/images/2.jfif'),
('Shirts', 'High Quality Shirts', 200.00, 25, '/images/3.jpg'),
('Jeans', 'High Quality Jeans', 200.00, 25, '/images/4.jfif');

SELECT * FROM products;
DELETE FROM order_items WHERE product_id < 7;

DELETE FROM products WHERE id < 7;
```



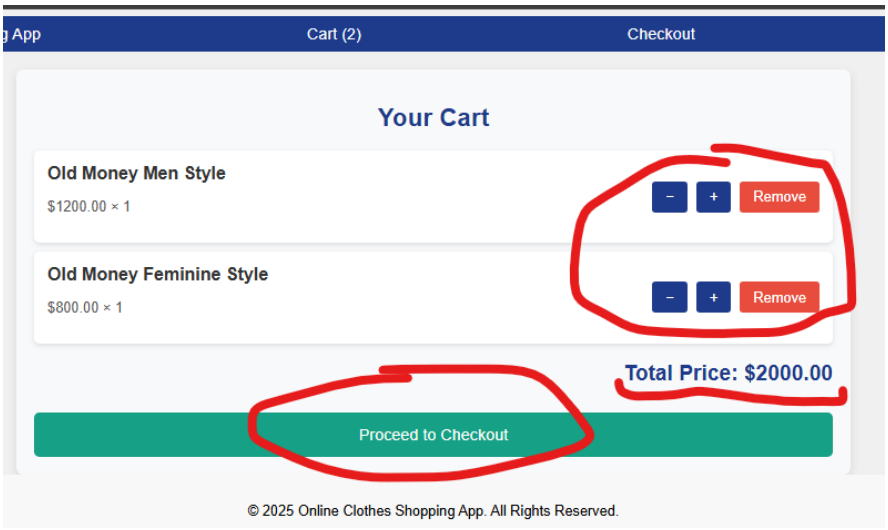
Backend (MySQL) has 4 products and frontend is successfully displaying the products!

Integration Testing 2: Checkout End-To-End:

Step	Component	Expected Output	Result
Add to cart	Frontend → JS	Cart updated	pass
Send checkout request	API → POST /order	Validated	pass
Deduct stock	MySQL	Updated stock	pass
Return success	Backend → UI	“Order Confirmed”	pass

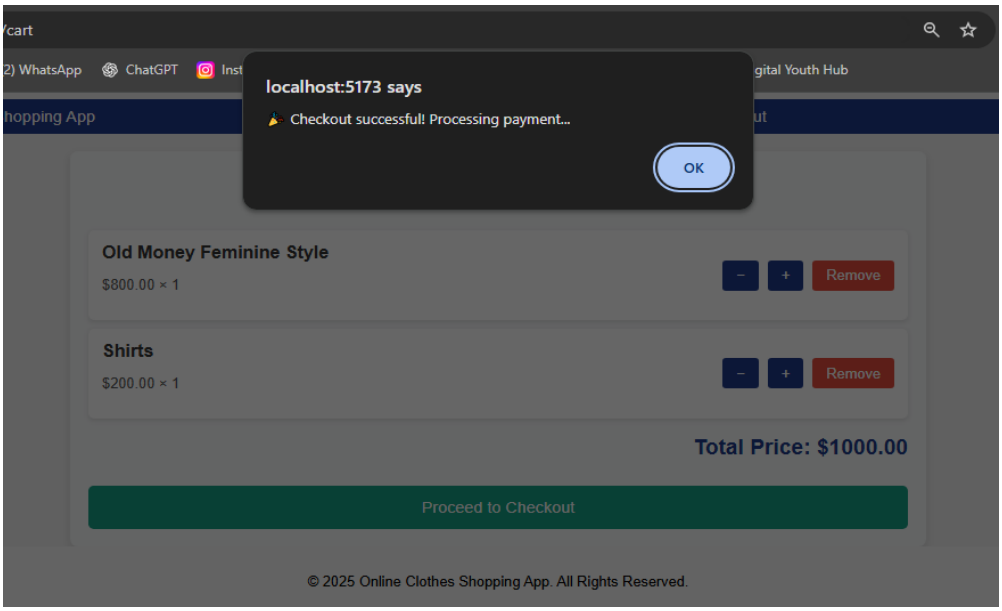


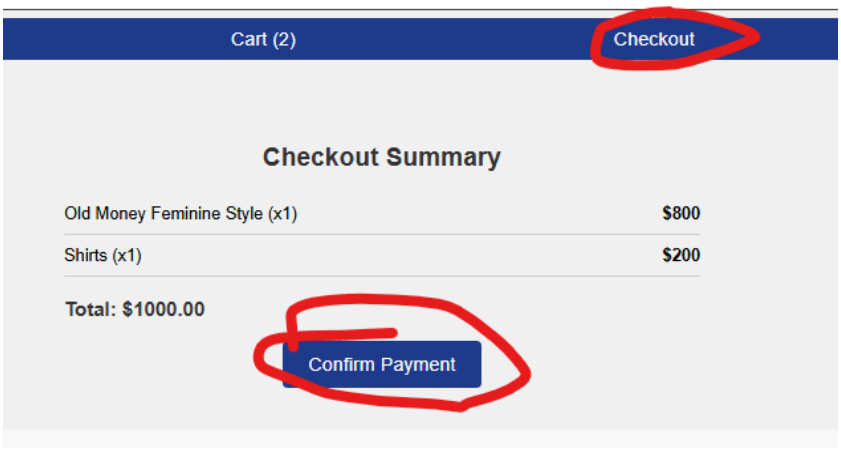
As customer adds 2 products into the cart so it will be incremented by 2.



This is the cart page, where you add(+), substract(-) or remove the whole product

I also click on the “Proceed to Checkout” button





This is checkout page where the selected products are now showing for the “payment confirmation”

CONCLUSION:

The Online Clothing Store Application was successfully tested using manual, unit, functional and integration testing. All major modules — product retrieval, cart management, checkout and stock updating — performed correctly. Exception handling ensured the system remained stable even when invalid data or unexpected errors occurred. No critical issues were found, and the application is ready for deployment.